B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019

Project Management, Entrepreneurship

23IS7PCPME

Project Report

ON

EV ChargeSight: Charging Infrastructure Management

EV Charger Operations and Maintenance

Submitted by

Pratham Manoj Patil (1BM23IS180)

Praveen Kumar Y (1BM23IS181)

Ravi Teja S (1BM23IS193)

Rishikesh Bulagouda (1BM23IS198)

in partial fulfilment of the requirements of the course Project Management, Entrepreneurship (23IS7PCPME) for the degree of

BACHELOR OF ENGINEERING

in

Department of Information Science and Engineering

Course Faculty

Shobana T S

Department of ISE,

B.M.S. College of Engineering

2026–2027

Acknowledgement

We express our sincere gratitude to **Shobana T S**, course faculty for Project Management, Entrepreneurship (23IS7PCPME), for her guidance on project planning, feasibility analysis and entrepreneurial thinking, which shaped the structure of this report.

We thank the Head of the Department and the faculty of the Department of Information Science and Engineering, B.M.S. College of Engineering, for the academic environment and the resources that made this work possible.

We acknowledge the open communities whose work this prototype builds on: the FastAPI, SQLAlchemy and Python ecosystems, the Open Charge Alliance for the openly published OCPP protocol, and the Open Charge Map project for public charging-station data released under a Creative Commons licence.

Finally, we thank our families and classmates for their encouragement and feedback during the development of ChargeSight.

The project team

Abstract

Public electric-vehicle (EV) charging in India is expanding quickly: more than 29,000 public charging stations had been installed by February 2026, and the PM E-DRIVE scheme earmarks ₹2,000 crore for further public charging infrastructure [5]–[7]. Studies of operating networks show that a sizeable share of public chargers can be out of service at any time [8], [9]. Small and mid-sized charge-point operators therefore need a dependable way to see which chargers are failing, decide which fault to repair first, and send a qualified technician within an agreed service level.

This report presents **EV ChargeSight**, a multi-organisation software-as-a-service (SaaS) prototype for EV charger operations and maintenance, and analyses it with the methods prescribed in the course Project Management, Entrepreneurship (23IS7PCPME). The implemented prototype provides a fleet dashboard, an event pipeline for simulated, imported and live (operator API and OCPP 1.6J) data, deduplicated maintenance tickets, an explainable priority score $P = 0.4S + 0.3U + 0.2R + 0.1A$, rule-based technician assignment, SLA escalation, analytics, public-station mapping, site-feasibility analysis, role-based access with tenant isolation, and a subscription catalogue with a clearly labelled test checkout. The repository contains 176 guarded API routes, 33 database tables and 386 automated tests, all of which passed in the latest run. Fleet data in the prototype is synthetic and labelled SIMULATED; no real operator is connected, and no payment has been taken.

The project-management analysis covers the project life cycle, pre-feasibility and feasibility appraisal, a sixteen-activity work breakdown structure, an organisation breakdown structure, CPM and PERT scheduling (a 47-working-day critical path of A–C–D–E–F–H–L–M–O–P, with a 77.7 % probability of finishing within 50 days), crashing and time-cost trade-off, a cost estimate of ₹1,80,990 for the first production year including contingency, monitoring and control with an MIS, and a risk register. The entrepreneurship analysis covers the Business Model Canvas, market and competitor analysis, innovation management, design thinking and TRIZ, a minimum viable product, and a commercial model whose hypothetical break-even point is 7 paying customers. All forecasts are explicitly hypothetical and require validation with real operators.

Keywords: EV charging, operations and maintenance, explainable prioritisation, CPM, PERT, Business Model Canvas, SaaS, OCPP.

Table of Contents

Acknowledgement ii

Abstract iii

List of Figures vi

List of Tables vi

List of Abbreviations vii

Chapter 1 Problem Identification 1

1.1 Background: growth of EV charging infrastructure 1

1.2 Operational challenges faced by charge-point operators 1

1.3 Problem statement 1

1.4 Stakeholders and their needs 2

1.5 Existing approaches and their limitations 2

1.6 Why a unified operations and maintenance platform is useful 2

Chapter 2 Objectives, Scope and Feasibility 2

2.1 Primary and specific objectives 2

2.2 Functional and non-functional requirements 3

2.3 Scope, boundaries, assumptions and constraints 4

2.4 Deliverables and acceptance criteria 4

2.5 ChargeSight as a project: the project life cycle 4

2.6 Pre-feasibility and feasibility appraisal 5

Chapter 3 Proposed Methodology 5

3.1 Data acquisition and source classification 5

3.2 Monitoring, fault detection and ticket creation 6

3.3 Explainable prioritisation 6

3.4 Assignment, SLA tracking, repair and analytics 6

Chapter 4 System Architecture and Technical Implementation 6

4.1 High-level architecture 6

4.2 Components and data flow 7

4.3 Implementation summary 8

4.4 Validation, error handling and audit 8

Chapter 5 Project Planning, Work Breakdown Structure and Activities 8

5.1 Importance and steps of project planning 8

5.2 Work Breakdown Structure 9

5.3 Organisation Breakdown Structure 10

5.4 Phased project planning 11

Chapter 6 Network Diagram, CPM and PERT Analysis 11

6.1 Activity-on-node network 11

6.2 CPM calculation 11

6.3 Critical path and its significance 13

6.4 Floats and slack 13

6.5 PERT analysis 13

6.6 Crashing and time-cost trade-off 14

Chapter 7 Gantt Chart and Project Timeline 14

Chapter 8 Resource Requirements 16

8.1 Human resources and allocation 16

8.2 Tools, computing, data and hosting 16

Chapter 9 Detailed Cost Estimation, Budget and Capital Budgeting 17

9.1 Costing approach 17
9.2 Development effort (notional) 17
9.3 One-time and recurring costs 17
9.4 Project cost-reduction methods 18
9.5 Capital budgeting (hypothetical) 18
9.6 Social cost-benefit analysis 19
Chapter 10 Project Monitoring and Control 19
10.1 Role of the project manager 19
10.2 Monitoring indicators and MIS 19
10.3 Quality assurance, change control and progress reporting 20
10.4 Project audit, acceptance and closure 20
10.5 Issues and problems in project management 20
Chapter 11 Risk Identification and Mitigation 21
11.1 Risk-opportunity perspective 22
Chapter 12 Business Model Canvas 22
12.1 Delivering measurable value without unverified claims 23
Chapter 13 Market, Industry and Customer Analysis 23
13.1 Market segment and target customers 23
13.2 Industry and competitor analysis 24
13.3 Factors determining competitive advantage 24
13.4 Customer discovery, adoption barriers, pilot and market entry 25
Chapter 14 Prototype (MVP) and Validation 25
14.1 Minimum viable product 25
14.2 Verified functionality versus planned functionality 26
14.3 Functional, security and tenant-isolation testing 26
14.4 Remaining gaps before commercial deployment 27
Chapter 15 Entrepreneurship, Commercial Viability and Pitch 27
15.1 Entrepreneurship and innovation in ChargeSight 27
15.2 Innovation management, idea management and thinking styles 27
15.3 Revenue model, pricing hypotheses and plan differentiation 28
15.4 Customer acquisition, partnerships, scaling and competitive risks 29
15.5 Idea incubation, design thinking and TRIZ 29
15.6 Entrepreneurial pitch 29
Chapter 16 Expected Outcomes, Conclusion and Future Scope 30
16.1 Expected outcomes 30
16.2 Conclusion 30
16.3 Current limitations and future scope 30
References 30
Appendix A Syllabus Coverage Map 31
Appendix B Verified Prototype Facts and Test Evidence 32

List of Figures

Figure 3.1 ChargeSight operational workflow (verified against the repository) 5
Figure 4.1 High-level architecture of ChargeSight 7
Figure 4.2 Component diagram: user-interface views, API routers and core services 7
Figure 4.3 Level-1 data-flow diagram 7
Figure 5.1 Work Breakdown Structure (critical work packages outlined in red) 9
Figure 5.2 Organisation Breakdown Structure (proposed role allocation) 10
Figure 6.1 AON project network with CPM values (durations in working days; critical path in red; project duration 47 days). Node: top row ES | activity (duration) | EF; bottom row LS | TF | LF 12
Figure 7.1 Gantt chart (planning schedule, 47 working days, 5 days per week) 15
Figure 9.1 Cost breakdown for the first production year (estimated, ASSUMPTION-based) 18
Figure 11.1 Risk heat map (likelihood × impact; red ≥ 15, orange 8–14, green < 8) 22
Figure 12.1 Business Model Canvas for ChargeSight (hypotheses to be validated) 23
Figure 14.1 MVP user journey 25
Figure 14.2 Ranked maintenance tickets with priority scores, technician, SLA and next action (real screenshot; SIMULATED data) 26
Figure 15.1 Break-even chart (hypothetical; based on proposed prices and an assumed plan mix) 28

List of Tables

Table 1.1 Stakeholders of ChargeSight and their needs 2
Table 2.1 Functional (FR) and non-functional (NFR) requirements with their status 3
Table 2.2 In-scope and out-of-scope items 4
Table 2.3 Project life-cycle phases applied to ChargeSight 4
Table 2.4 Feasibility appraisal summary (Unit 2) 5
Table 4.1 Technical implementation (verified in the repository) 8
Table 5.1 WBS activity table (durations in working days; critical activities in red) 9
Table 6.1 CPM schedule: earliest/latest times and floats (working days) 11
Table 6.2 PERT estimates for critical-path activities (working days) 13
Table 6.3 Crashing data (notional effort cost, ASSUMPTION) 14
Table 6.4 Time-cost trade-off: cheapest crash plan for each duration (exhaustive search) 14
Table 7.1 Milestones 15
Table 8.1 Resource-allocation table (lead effort by role) 16
Table 8.2 Existing resources and additional resources needed for production 16
Table 9.1 One-time production costs 17
Table 9.2 Recurring operating costs 17
Table 9.3 First-year production budget 18
Table 9.4 Hypothetical three-year cash flows (₹; discount rate 12 %) 19
Table 10.1 Project-monitoring table 20
Table 11.1 Risk register 21
Table 13.1 Buyer and user personas (illustrative, not interview results) 24
Table 13.2 Existing alternatives and positioning (based on public product information) 24
Table 14.1 MVP feature status 26
Table 15.1 Steps of innovation management applied to ChargeSight 27
Table 15.2 Proposed subscription catalogue as implemented in the prototype (proposed prices, not validated) 28

List of Abbreviations

AC / DC	Alternating current / direct current
AON	Activity on node
API	Application programming interface
ARPU	Average revenue per user (customer)
BMC	Business Model Canvas
CPM	Critical Path Method
CPO	Charge-point operator
CPMS	Charge-point management system
CSRF	Cross-site request forgery
CSV	Comma-separated values
DFD	Data-flow diagram
DPR	Detailed project report
EF / ES	Earliest finish / earliest start
EV	Electric vehicle
FR / NFR	Functional / non-functional requirement
GST	Goods and Services Tax
IRR	Internal rate of return
LF / LS	Latest finish / latest start
MIS	Management information system
MTTA / MTTR	Mean time to acknowledge / mean time to repair
MVP	Minimum viable product
NPV	Net present value
OBS	Organisation breakdown structure
OCM	Open Charge Map
OCPP	Open Charge Point Protocol
ORM	Object-relational mapping
PERT	Programme Evaluation and Review Technique
PM E-DRIVE	PM Electric Drive Revolution in Innovative Vehicle Enhancement scheme
SaaS	Software as a service
SCBA	Social cost-benefit analysis
SLA	Service-level agreement
SPA	Single-page application
TF / FF	Total float / free float
TRIZ	Theory of Inventive Problem Solving
UPI	Unified Payments Interface
WBS	Work breakdown structure

CHAPTER 1

Problem Identification

1.1 Background: growth of EV charging infrastructure

India's public EV charging network is growing quickly. In a written reply in the Lok Sabha, the Minister of State for Heavy Industries reported that more than 29,000 public EV charging stations had been installed across the country by February 2026, with more than ₹912 crore committed under FAME-II and ₹2,000 crore allocated under PM E-DRIVE for public charging infrastructure [5]. Under PM E-DRIVE, the ₹2,000 crore outlay is intended to support about 72,000 public charging stations along 50 national-highway corridors and at high-traffic locations such as metro cities, toll plazas, railway stations, airports and fuel outlets [6]. The scheme subsidises infrastructure and chargers at rates of up to 100 % for government premises and 70–80 % at transport hubs and other public sites [7].

Installing chargers is only the first half of the problem. Each charger is a field asset with power electronics, connectors, payment readers, network modems and software, and every one of these can fail. Evidence from operating networks shows how often this happens. A field survey of 657 open public DC fast-charging connectors in the San Francisco Bay Area found that 22.7 % of the chargers were non-functional, because of unresponsive screens, payment failures, charge-initiation failures, network failures or broken connectors [8]. J.D. Power's 2025 U.S. EV Experience Public Charging Study reported that 14 % of EV owners had visited a charger without successfully charging, an improvement of five percentage points over 2024 [9]. These studies are from the United States and are cited here only to show that reliability is a recognised operational problem; comparable Indian survey data was not available to the team.

1.2 Operational challenges faced by charge-point operators

From the stakeholders' perspective and the published evidence, the team identified six recurring operational challenges. They are stated here as working hypotheses to be validated through customer discovery (Chapter 13), not as survey findings.

- **Fragmented monitoring.** Charger status, faults and session data often live in vendor portals, spreadsheets and messaging groups, so no single view shows the health of the whole fleet.
- **Delayed fault identification.** A charger that silently stops sending heartbeats may stay unnoticed until a driver complains.
- **No common basis for prioritisation.** When several chargers fail at once, the repair order depends on whoever reports loudest, not on severity, site demand or the capacity lost at a site.
- **Technician allocation.** Faults need different skills (payment, electrical safety, communication) and AC or DC equipment, and technicians have limited capacity and shifts.
- **SLA management.** Without deadlines attached to each fault, overdue repairs are discovered late, and response and repair times (MTTA and MTTR) are not measured.
- **Limited reporting and planning.** Operators lack trustworthy history for analysing fault trends, comparing sites or deciding where to invest in new chargers.

1.3 Problem statement

> **Problem statement.** Small and mid-sized EV charge-point operators lack an affordable, unified system that detects charger faults from any data source, turns each fault into a single accountable maintenance ticket, ranks tickets with a transparent and defensible priority rule, assigns a qualified and available technician within a service level, and turns the resulting history into operational and investment decisions.

1.4 Stakeholders and their needs

Table 1.1: Stakeholders of ChargeSight and their needs

Stakeholder	Primary need	How ChargeSight responds (implemented)
Charge-point operator (owner / operations manager)	Fleet availability, fewer lost sessions, accountable repairs	Dashboard, ranked ticket queue, SLA and MTTA/MTTR metrics
Field technician	Clear job, location, fault details, safety steps	Assigned tickets, lifecycle actions, notes, safety inspection before return to service
EV driver (indirect)	A working charger when they arrive	Faster detection and repair; user-reported tickets
Site host (mall, office, fuel outlet)	Reliable service at their premises	Site-level availability and downtime reporting
Platform administrator	Secure multi-tenant operation, plans, support	Roles, tenant isolation, plan catalogue, audit log
Course faculty / evaluators	Sound project-management and entrepreneurship analysis	This report, the prototype and the test evidence

1.5 Existing approaches and their limitations

Operators today generally rely on one or more of three approaches. **Vendor or charge-point management system (CPMS) portals**, such as AMPECO and Driivz internationally and Kazam in India, provide remote monitoring, configuration and reporting for the chargers they manage [13]–[15]. **Manual workflows** (spreadsheets, phone calls and messaging groups) are cheap, but leave no audit trail and no prioritisation logic. **General-purpose ticketing or field-service tools** track work orders, but do not understand charger status, fault categories or OCPP events. Chapter 13 compares these approaches in more detail; the publicly available material reviewed does not describe an explainable, factor-by-factor repair-priority score combined with technician-eligibility rules and SLA escalation in one affordable package for small operators.

1.6 Why a unified operations and maintenance platform is useful

A unified platform puts the whole chain of detection, ticket, priority, assignment, repair, verification and reporting into one data model, with an audit trail at every step. This makes the operator's performance measurable (uptime, MTTA, MTTR, overdue tickets), makes repair decisions explainable to site hosts and management, and creates the historical evidence needed for planning new sites. ChargeSight was conceived as a simulation-first prototype of such a platform, so that the full workflow could be built and tested without access to physical chargers or live operator data, as the course required.

CHAPTER 2

Objectives, Scope and Feasibility

2.1 Primary and specific objectives

Primary objective. To design, build and validate a working prototype of a multi-organisation SaaS platform that helps EV charge-point operators monitor charger availability, manage faults through prioritised and accountable maintenance tickets, and analyse operational and investment performance, and to plan its development and commercialisation using the methods of the course.

Specific objectives.

1. Ingest charger events from four sources (simulator, CSV import, operator REST API and OCPP 1.6J), keeping the provenance label of every record.
2. Detect faults and offline chargers and create exactly one open ticket per charger and fault category.
3. Rank open tickets with an explainable weighted score, and prevent low-priority faults from being starved indefinitely.
4. Assign technicians using explicit eligibility rules (skill, AC/DC equipment, safety certification, availability, shift and capacity).
5. Track SLA deadlines by severity, escalate overdue tickets once, and report MTTA and MTTR.
6. Provide analytics, exports, public-station mapping and site-feasibility analysis.
7. Enforce authentication, role-based access, tenant isolation and plan entitlements on the server.
8. Produce the project-management and business analysis required by the syllabus (WBS, CPM/PERT, Gantt, cost, risk, Business Model Canvas, MVP and pitch).

2.2 Functional and non-functional requirements

Table 2.1: Functional (FR) and non-functional (NFR) requirements with their status

ID	Requirement	Status
FR1	Fleet dashboard showing charger status counts, uptime, power, energy, sessions and open tickets per data source	Implemented
FR2	Event ingestion with transition validation (simulator, CSV, operator API, OCPP 1.6J)	Implemented; live sources tested with mocks and a local OCPP handshake
FR3	Ticket lifecycle Open → Assigned → En Route → On Site → Resolved → Closed, with waiting states and reopening	Implemented
FR4	Explainable priority score and ranked queue	Implemented
FR5	Automatic and manual technician assignment with eligibility reasons	Implemented
FR6	SLA deadlines (4 h / 12 h / 48 h assumptions) and single escalation	Implemented
FR7	Analytics, CSV exports and printable summary	Implemented
FR8	Public charging-station map and site-feasibility tool	Implemented (map with Open Charge Map snapshot; Google Maps optional)
FR9	Organisations, roles, invitations, plans, entitlements and test checkout	Implemented (no real payment gateway)
NFR1	Security: hashed passwords (Argon2id), CSRF protection, deny-by-default route guard	Implemented and tested
NFR2	Tenant isolation: no organisation can read another organisation's data	Implemented and tested
NFR3	Honesty of data: every record labelled SIMULATED, IMPORTED or LIVE	Implemented
NFR4	Reliability: verified database backups before migrations; idempotent seeding	Implemented
NFR5	Usability: responsive layout, accessible labels, keyboard focus	Implemented; no formal usability study yet
NFR6	Scalability to thousands of chargers	Proposed (requires PostgreSQL and load testing)

2.3 Scope, boundaries, assumptions and constraints

Table 2.2: In-scope and out-of-scope items

In scope (this project)	Out of scope (now) / proposed
Software prototype running on a laptop or one server	Charger hardware, installation and electrical work
Synthetic 40-charger fleet across 6 Bengaluru demonstration areas	Real operator data (needs written authorisation and credentials)
Integration pathways tested with mocks and a local OCPP simulator	Certified OCPP conformance testing
Subscription catalogue and labelled TEST checkout	Real payment gateway, GST invoicing
Explainable rule-based prioritisation	Machine-learning failure prediction (future scope)
Project plan, cost estimate and business model	Fund-raising, company registration

Assumptions. The priority weights (0.4/0.3/0.2/0.1), the SLA hours and the technician travel speed are design assumptions, labelled ASSUMPTION in the software. The team works about four hours per member per working day alongside other coursework, and free tiers of developer tools are sufficient for the prototype. **Constraints.** There is no access to physical chargers, which the faculty required to keep the MVP simulation-only. Operator API access has been requested from an industry contact but not granted. The team has four members, and the plan allows about nine to ten working weeks.

2.4 Deliverables and acceptance criteria

The deliverables are: (1) the working prototype with source code and documentation; (2) an automated test suite; (3) this project report, which also serves as the detailed project report (DPR); and (4) a presentation with a live demonstration and an entrepreneurial pitch. The acceptance criteria are that every functional requirement marked "Implemented" is demonstrable in the Presentation Demo workspace, that the automated test suite passes with no failures or skipped tests, that no screen labels simulated data as live, and that the plan, costs and business analysis in this report are internally consistent.

2.5 ChargeSight as a project: the project life cycle

A *project* is a temporary endeavour with a defined beginning and end, undertaken to create a unique product or service within constraints of time, cost and quality; *project management* is the application of planning, organising, scheduling, monitoring and control to achieve that objective [1], [3]. ChargeSight meets this definition: it has a fixed academic timeline, a unique deliverable and a constrained team and budget. Table 2.3 maps the four life-cycle phases from the syllabus onto the project.

Table 2.3: Project life-cycle phases applied to ChargeSight

Phase	Key activities in ChargeSight	Phase output
Initiation / conceptualisation	Problem identification, stakeholder needs, project charter, pre-feasibility (activities A–D)	Approved concept and feasibility report
Planning	Scope, WBS, OBS, CPM/PERT schedule, cost estimate, risk register (this report)	Baseline plan
Implementation / execution	Design, backend, frontend, ticketing, analytics, integrations, SaaS layer, testing (E–M)	Tested prototype
Closure / termination	Documentation, final presentation, project audit, handover of repository (O–P)	Report, pitch, lessons learned

2.6 Pre-feasibility and feasibility appraisal

A **pre-feasibility study** screened the idea before detailed work began. It confirmed that the problem is real and supported by public evidence (Section 1.1), that a simulation-first software MVP is technically achievable by the team, and that subscription software has low capital requirements. The detailed **feasibility study** then appraised the project along the dimensions prescribed in Unit 2, summarised in Table 2.4. The capital-budgeting figures are calculated in Chapter 9.

Table 2.4: Feasibility appraisal summary (Unit 2)

Appraisal	Question asked	Finding
Technical appraisal	Can the system be built and operated with available skills and tools?	Feasible. Python/FastAPI/SQLite stack; prototype built and 386 automated tests pass. Production needs PostgreSQL, hosting and a security review.
Economic appraisal	Do the benefits to the economy justify the resources?	Positive in principle. Higher charger availability supports public investment in charging (PM E-DRIVE). Not quantified with field data.
Commercial appraisal	Is there a market and a way to reach it?	Plausible but unvalidated. Over 29,000 public stations and many small operators; customer discovery and a pilot are required (Chapter 13).
Financial appraisal	Will revenues cover costs?	Hypothetical break-even at 7 paying customers; low one-time investment (₹76,000); see Chapter 9.
Social cost-benefit	What are the wider costs and benefits?	Benefits: fewer failed charging visits, lower emissions through EV adoption, skilled jobs. Costs: data-privacy and security obligations (Section 9.6).

CHAPTER 3

Proposed Methodology

ChargeSight follows a ten-stage operational workflow, shown in Figure 3.1. Every stage was checked against the repository. Stages 1–10 are implemented in the prototype; the only limitation is that live sources (operator API and OCPP) have been tested with mocks and a local OCPP simulator, not with a real operator.

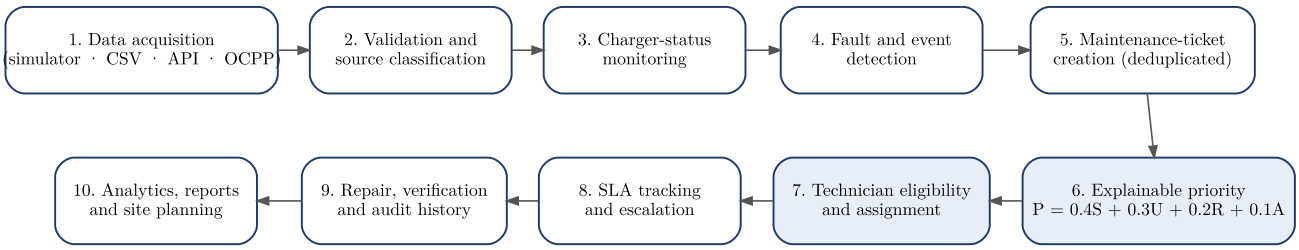


Figure 3.1: ChargeSight operational workflow (verified against the repository)

3.1 Data acquisition and source classification

Stage 1 acquires charger events from four pathways: the seeded simulator (40 chargers, 15-second steps, OCPP 1.6-style messages on an internal event bus), CSV import of an authorised operator export, an operator REST API adapter, and an OCPP 1.6J WebSocket central system. **Stage 2** normalises every

event into a common `ExternalEvent` structure, removes duplicates, validates status transitions and stores a provenance label: SIMULATED, IMPORTED or LIVE. The user interface never mixes sources: each view shows one data source at a time.

3.2 Monitoring, fault detection and ticket creation

Stage 3 keeps the current status of every charger (available, charging, faulted, offline or unavailable); only one service module is allowed to change charger status. **Stage 4** raises a fault when a charger reports one, or marks it offline after three missed heartbeats. **Stage 5** creates a maintenance ticket; a database index guarantees at most one unresolved ticket per charger and fault category, so repeated events do not create duplicate work. Users can also raise tickets manually. For resettable faults a remote reset is attempted first, and a successful reset closes the ticket automatically.

3.3 Explainable prioritisation

Stage 6 ranks every open ticket with a weighted score whose factors all lie between 0 and 1:

$$P = 0.4\ S + 0.3\ U + 0.2\ R + 0.1\ A$$

Here S is severity (critical 1.0, major 0.6, minor 0.2; safety faults are always critical), U is the site's expected utilisation relative to the busiest site, R is the share of the site's working capacity lost (1 when the last working charger fails), and A is fault age, capped at 24 hours. The weights are a documented design assumption, not an industry standard, and administrators can change them. Tickets that have waited 24 hours or more are escalated to the top regardless of score, so low-priority faults cannot be starved. Every ticket shows its individual factor contributions, which makes the ranking explainable to managers and site hosts.

3.4 Assignment, SLA tracking, repair and analytics

Stage 7 assigns a technician only if all eligibility rules hold: the technician is active, available and on shift, has the required skill (for example, safety for ground faults), has DC equipment for DC chargers, is safety-certified for safety faults, and is below their workload capacity. Among eligible technicians the choice is deterministic (lower workload first, then distance). If nobody is eligible, the reasons are stored and the ticket is flagged *dispatch needed*. **Stage 8** sets an SLA deadline of 4, 12 or 48 hours by severity and escalates an overdue ticket exactly once. **Stage 9** records every status change, assignment, note and inspection in the ticket history; a safety fault returns to service only after a post-repair inspection has been recorded. **Stage 10** turns the stored history into uptime trends, fault-frequency charts, MTTA/MTTR, site comparisons, CSV exports, a FIFO-versus-priority policy laboratory and a site-feasibility model.

CHAPTER 4

System Architecture and Technical Implementation

4.1 High-level architecture

ChargeSight is a single deployable web application (Figure 4.1). A browser loads either the public website or the operations console, a single-page application written in plain JavaScript ES modules with hand-drawn SVG charts and no build step. Both talk to a FastAPI backend over JSON. Every request first passes a security guard that checks the session, the CSRF token, the user's role and the

organisation's plan before any router code runs. Routers delegate to service modules, which use pure, unit-tested core models (priority, policy simulation, finance and planning) and persist data through SQLAlchemy 2.0 to SQLite in WAL mode.

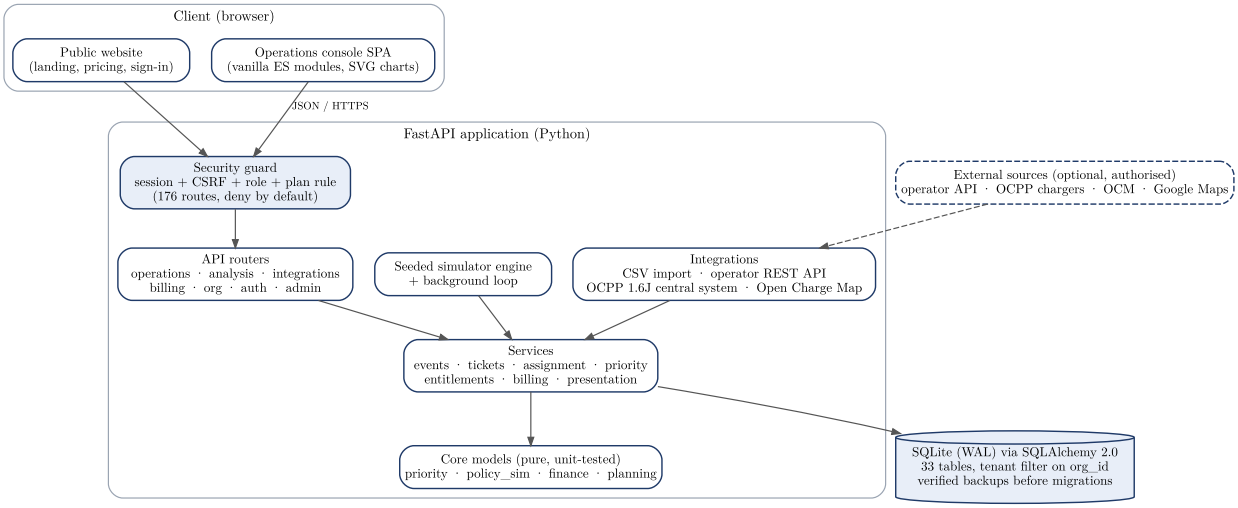


Figure 4.1: High-level architecture of ChargeSight

4.2 Components and data flow

Figure 4.2 shows the main components and their dependencies. Three components are central to the design: `events.py` is the only module allowed to change a charger's status; `priority_service.py` builds the inputs to the priority engine from the database; and `tenancy.py` attaches an organisation filter to every query. Figure 4.3 is a level-1 data-flow diagram from external sources to reports.

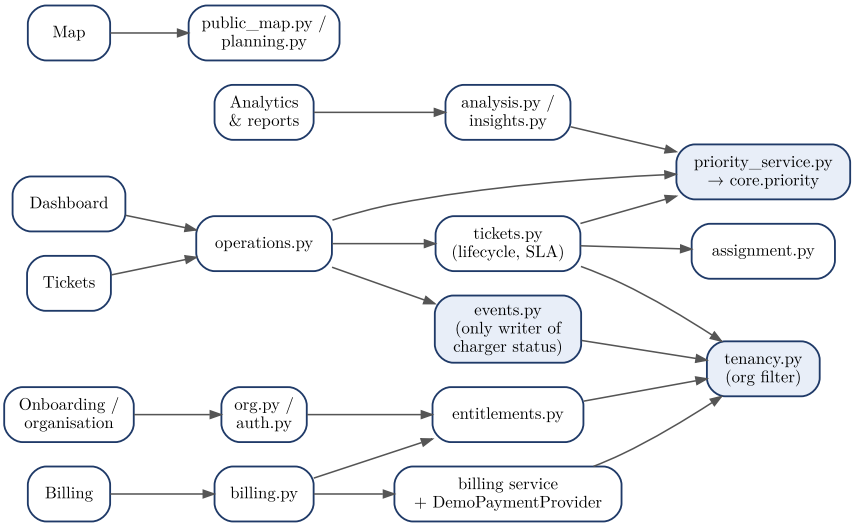


Figure 4.2: Component diagram: user-interface views, API routers and core services

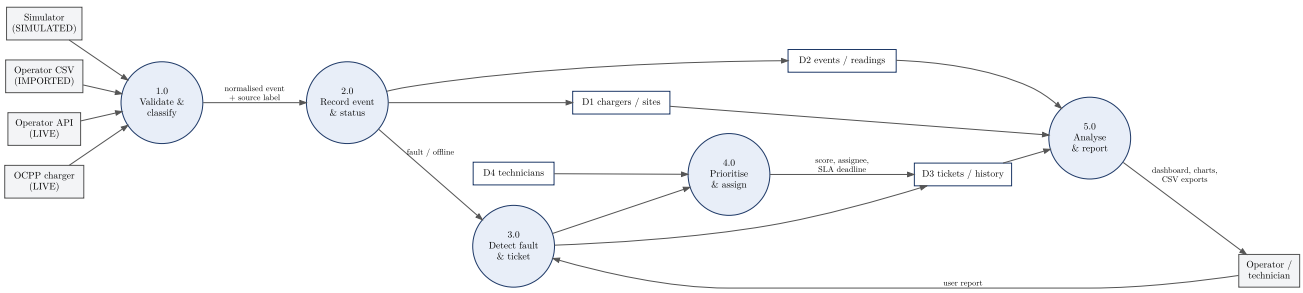


Figure 4.3: Level-1 data-flow diagram

4.3 Implementation summary

Table 4.1: Technical implementation (verified in the repository)

Area	Implementation
Frontend	Vanilla ES-module SPA (about 5,900 lines of JavaScript); 16 console routes; public site with pricing calculator; responsive layout
Backend API	FastAPI with Pydantic v2 validation; 176 routes, each with an explicit guard rule (deny by default); about 16,100 lines of Python
Database	SQLite (WAL) through SQLAlchemy 2.0; 33 tables; check constraints on sources and statuses; forward-only migrations preceded by verified backups
Authentication	Argon2id password hashing, HttpOnly session cookies, CSRF tokens, sign-in rate limiting, password reset and e-mail verification (development outbox)
Authorisation	Five roles (owner, admin, operations manager, technician, viewer) mapped to a least-privilege permission matrix
Tenant isolation	`org_id` on every tenant table; the ORM adds the organisation filter to every SELECT, UPDATE and DELETE; new rows inherit the session organisation
Ticketing and priority	Validated lifecycle, audit history, SLA sweep, recurrence reopening; priority engine as a pure module
Integrations	CSV import, operator REST adapter, OCPP 1.6J central system, Open Charge Map snapshot; credentials encrypted (Fernet) and redacted from logs; outbound requests checked against server-side request forgery (SSRF)
Subscriptions	Central plan catalogue, server-side Decimal pricing, entitlements enforced on creation, import, API sync, OCPP and invitations; TEST-only checkout
Quality	386 automated tests, including 20 real-browser tests (Playwright)

4.4 Validation, error handling and audit

Requests are validated by Pydantic models, and errors return readable messages with appropriate HTTP status codes (404, 409, 422). Status transitions outside the allowed set are rejected unless an administrator overrides them, and every override is audited. Organisation-level actions (sign-in, plan changes, integration changes, workspace changes) are written to an audit log. Simulated, imported and live records carry their provenance label from ingestion to export, and every model output is labelled ESTIMATED or ASSUMPTION, so prototype figures cannot be mistaken for measured results.

CHAPTER 5

Project Planning, Work Breakdown Structure and Activities

5.1 Importance and steps of project planning

Planning converts the objectives into an agreed scope, a schedule, a budget and a set of responsibilities against which progress can be measured [1], [3]. For ChargeSight the planning steps were: (1) define the objectives and scope (Chapter 2); (2) decompose the work into a WBS; (3) assign responsibility through an OBS; (4) estimate durations and dependencies; (5) schedule with CPM and PERT and draw the Gantt chart (Chapters 6–7); (6) estimate resources and costs (Chapters 8–9); (7) identify risks (Chapter 11); and (8) set the baseline used for monitoring and control (Chapter 10).

5.2 Work Breakdown Structure

The WBS (Figure 5.1) decomposes the project into four phases and sixteen work packages. The work packages map one-to-one onto the activities A–P used in the network diagram and the Gantt chart, so all three views agree. Table 5.1 lists each work package with its deliverable, responsible role, duration and predecessors.

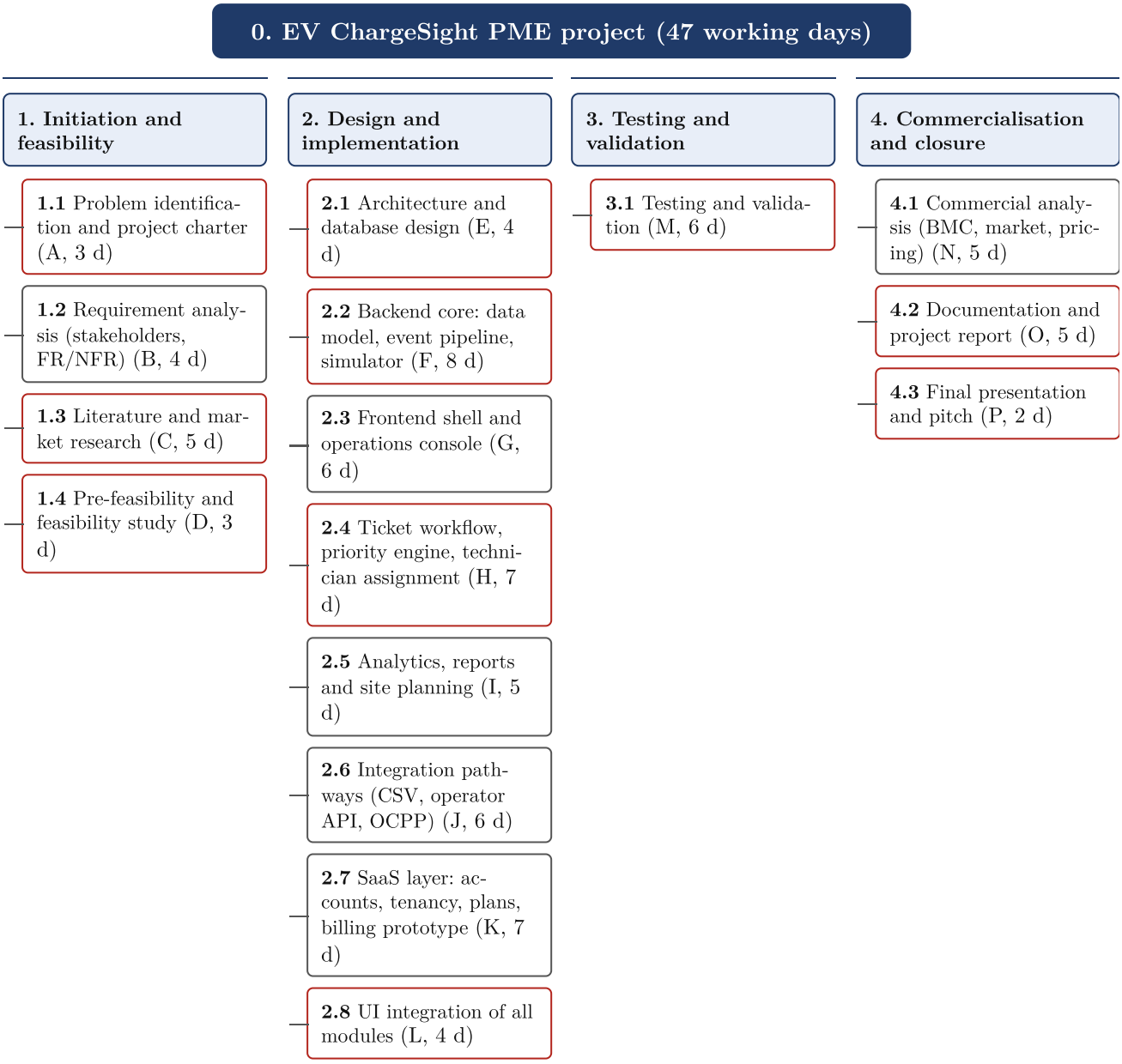


Figure 5.1: Work Breakdown Structure (critical work packages outlined in red)

Table 5.1: WBS activity table (durations in working days; critical activities in red)

WBS	ID	Activity	Deliverable	Responsible role	Days	Predecessors
1.1	**A**	**Problem identification and project charter**	**Problem statement, charter**	**Project manager**	**3**	—
1.2	B	Requirement analysis (stakeholders, FR/NFR)	Requirements specification	Business analyst and QA lead	4	A
1.3	**C**	**Literature and market research**	**Research notes, sources**	**Business analyst and QA lead**	**5**	**A**

WBS	ID	Activity	Deliverable	Responsible role	Days	Predecessors
1.4	**D**	**Pre-feasibility and feasibility study**	**Feasibility report**	**Project manager**	**3**	**B, C**
2.1	**E**	**Architecture and database design**	**Architecture, data model**	**Backend lead**	**4**	**D**
2.2	**F**	**Backend core: data model, event pipeline, simulator**	**API, simulator, seed data**	**Backend lead**	**8**	**E**
2.3	G	Frontend shell and operations console	Console UI, navigation	Frontend and UX lead	6	E
2.4	**H**	**Ticket workflow, priority engine, technician assignment**	**Ticketing and dispatch module**	**Backend lead**	**7**	**F**
2.5	I	Analytics, reports and site planning	Charts, exports, feasibility tool	Frontend and UX lead	5	F
2.6	J	Integration pathways (CSV, operator API, OCPP)	Connectors with tests	Backend lead	6	F
2.7	K	SaaS layer: accounts, tenancy, plans, billing prototype	Auth, roles, plans, demo checkout	Backend lead	7	F
2.8	**L**	**UI integration of all modules**	**Integrated console**	**Frontend and UX lead**	**4**	**G, H, I**
3.1	**M**	**Testing and validation**	**Test suite, defect log**	**Business analyst and QA lead**	**6**	**H, I, J, K, L**
4.1	N	Commercial analysis (BMC, market, pricing)	Business model, pricing hypotheses	Project manager	5	D
4.2	**O**	**Documentation and project report**	**Report, user guide**	**Project manager**	**5**	**M, N**
4.3	**P**	**Final presentation and pitch**	**Slides, demo, pitch**	**Project manager**	**2**	**O**

5.3 Organisation Breakdown Structure

The OBS (Figure 5.2) assigns each work package to one accountable role in a four-person team. The role allocation is proposed: it reflects each activity's main skill and will be confirmed by the team; every member contributes to testing and documentation.

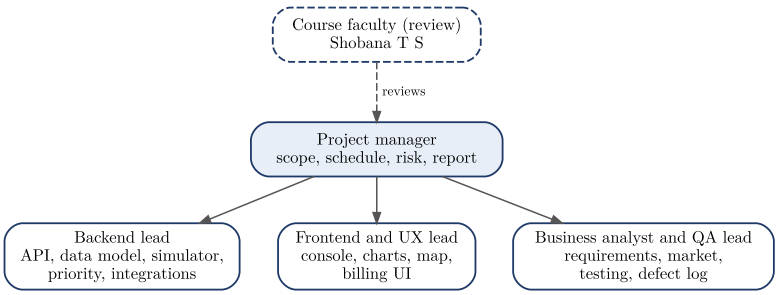


Figure 5.2: Organisation Breakdown Structure (proposed role allocation)

5.4 Phased project planning

The plan is phased so that each phase ends with a reviewable milestone: M1 feasibility approved (end of day 11), M2 design baselined (day 15), M3 integrated build (day 34), M4 tested release (day 40) and M5 final pitch (day 47). Beyond the academic project, commercialisation is also phased: Phase I is the validated prototype (this project); Phase II is a single-operator pilot with real data; Phase III is a paid launch for small operators; and Phase IV is scale-up, with PostgreSQL, monitoring and partnerships (Chapter 15).

CHAPTER 6

Network Diagram, CPM and PERT Analysis

6.1 Activity-on-node network

The project network is drawn as an activity-on-node (AON) diagram (Figure 6.1), using the sixteen activities and precedence relations in Table 5.1. Each node shows the earliest start (ES) and earliest finish (EF) on the top row, and the latest start (LS), total float (TF) and latest finish (LF) on the bottom row. Critical activities and links are drawn in red.

6.2 CPM calculation

Forward pass. Starting with ES = 0 for activity A, each activity's earliest start is the largest earliest finish of its predecessors. For example, D can start only after both B (EF = 7) and C (EF = 8) finish, so $ES_D = 8$. M depends on H, I, J, K and L, whose earliest finishes are 30, 28, 29, 30 and 34, so $ES_M = 34$. The last activity, P, finishes at day 47, which is the project duration.

The forward pass gives ES = max(EF of predecessors) and EF = ES + duration; the backward pass gives LF = min(LS of successors) and LS = LF − duration; total float TF = LS − ES.

Backward pass. Starting with LF = 47 for P and moving backwards, each activity's latest finish is the smallest latest start of its successors. Activities with zero total float form the **critical path**.

Table 6.1: CPM schedule: earliest/latest times and floats (working days)

ID	Activity	Dur.	ES	EF	LS	LF	TF	FF	Critical
A	**Problem identification and project charter**	**3**	**0**	**3**	**0**	**3**	**0**	**0**	**Yes**
B	Requirement analysis (stakeholders, FR/NFR)	4	3	7	4	8	1	1	No
C	**Literature and market research**	**5**	**3**	**8**	**3**	**8**	**0**	**0**	**Yes**
D	**Pre-feasibility and feasibility study**	**3**	**8**	**11**	**8**	**11**	**0**	**0**	**Yes**
E	**Architecture and database design**	**4**	**11**	**15**	**11**	**15**	**0**	**0**	**Yes**
F	**Backend core: data model, event pipeline, simulator**	**8**	**15**	**23**	**15**	**23**	**0**	**0**	**Yes**
G	Frontend shell and operations console	6	15	21	24	30	9	9	No
H	**Ticket workflow, priority engine, technician assignm...**	**7**	**23**	**30**	**23**	**30**	**0**	**0**	**Yes**
I	Analytics, reports and site planning	5	23	28	25	30	2	2	No

ID	Activity	Dur.	ES	EF	LS	LF	TF	FF	Critical
J	Integration pathways (CSV, operator API, OCPP)	6	23	29	28	34	5	5	No
K	SaaS layer: accounts, tenancy, plans, billing protot…	7	23	30	27	34	4	4	No
L	**UI integration of all modules**	**4**	**30**	**34**	**30**	**34**	**0**	**0**	**Yes**
M	**Testing and validation**	**6**	**34**	**40**	**34**	**40**	**0**	**0**	**Yes**
N	Commercial analysis (BMC, market, pricing)	5	11	16	35	40	24	24	No
O	**Documentation and project report**	**5**	**40**	**45**	**40**	**45**	**0**	**0**	**Yes**
P	**Final presentation and pitch**	**2**	**45**	**47**	**45**	**47**	**0**	**0**	**Yes**

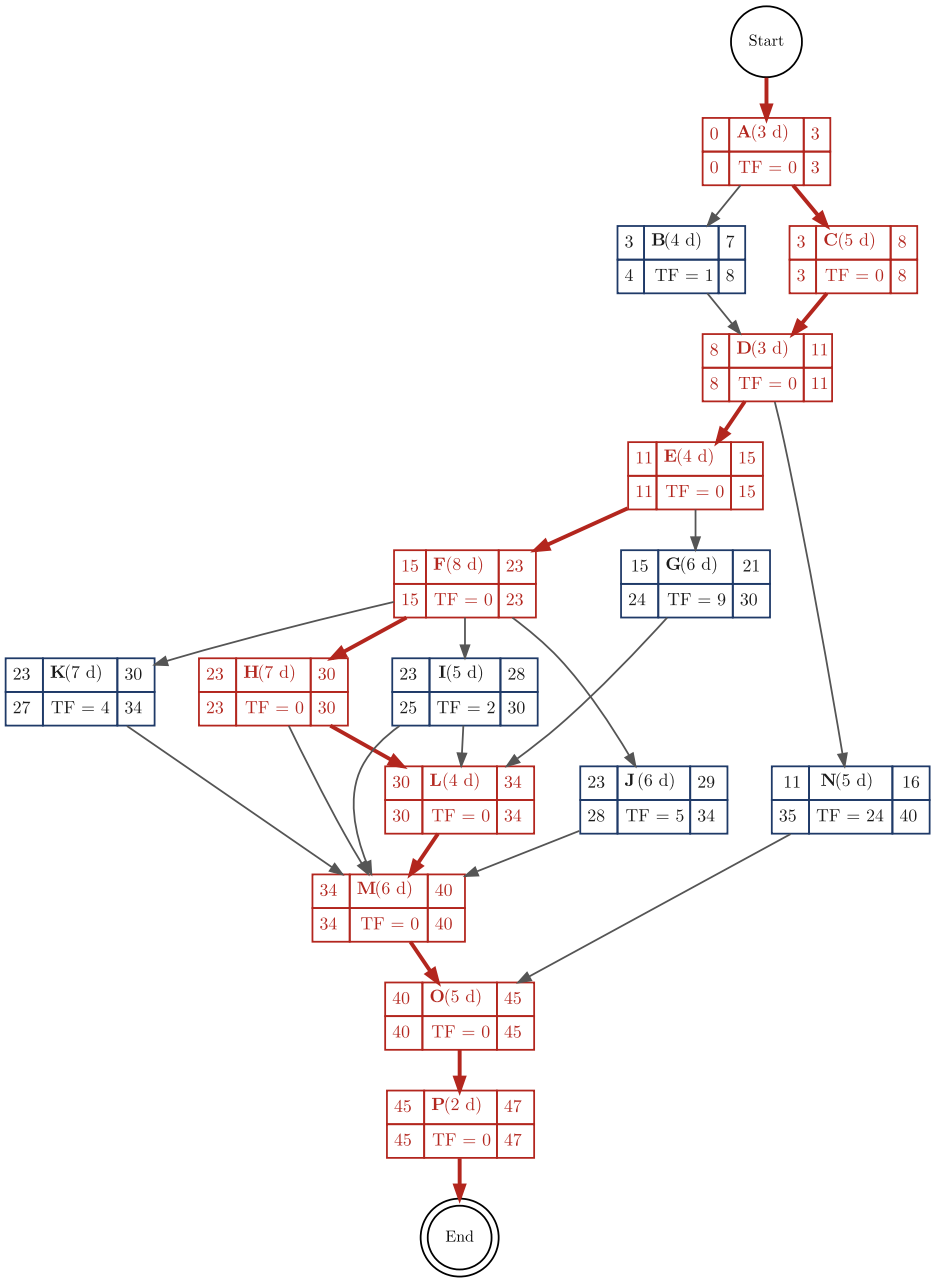


Figure 6.1: AON project network with CPM values (durations in working days; critical path in red; project duration 47 days). Node: top row ES | activity (duration) | EF; bottom row LS | TF | LF

6.3 Critical path and its significance

The critical path is **A → C → D → E → F → H → L → M → O → P**, with a length of 3 + 5 + 3 + 4 + 8 + 7 + 4 + 6 + 5 + 2 = **47 working days** (about 9.4 weeks). Any delay to a critical activity delays the final presentation by the same amount. The path runs through the backend core (F) and the ticketing and priority module (H), so these two activities received the strongest developer and the closest monitoring. Research (C, 5 days) rather than requirements (B, 4 days) is critical in the initiation phase, so the team started the literature and market research on the first day after the charter.

6.4 Floats and slack

Total float is the delay an activity can absorb without delaying the project; **free float** is the delay it can absorb without delaying any successor. In this plan the free float equals the total float for every activity, because each non-critical activity feeds a critical activity directly. Commercial analysis (N) has the largest float (24 days), so it can run in parallel with development and be scheduled flexibly around coursework. The frontend shell (G, 9 days), integrations (J, 5 days) and SaaS layer (K, 4 days) also have useful slack. Requirement analysis (B) has only 1 day of float and is nearly critical; it was monitored as if it were critical.

6.5 PERT analysis

Because development durations are uncertain, three time estimates were made for each activity: optimistic (o), most likely (m) and pessimistic (p). PERT uses the expected time $t_e = (o + 4m + p) / 6$ and the variance $\sigma^2 = ((p - o) / 6)^2$. Table 6.2 applies this to the critical-path activities.

Table 6.2: PERT estimates for critical-path activities (working days)

Activity	o	m	p	t_e	σ^2
A	2	3	4	3.00	0.111
C	4	5	7	5.17	0.250
D	2	3	5	3.17	0.250
E	3	4	6	4.17	0.250
F	6	8	12	8.33	1.000
H	5	7	10	7.17	0.694
L	3	4	6	4.17	0.250
M	4	6	9	6.17	0.694
O	4	5	7	5.17	0.250
P	1	2	3	2.00	0.111
Total				**48.50**	**3.861**

The expected project duration is $T_e = 48.50$ days, with $\sigma = \sqrt{3.861} = 1.96$ days. The probability of finishing within 50 working days is $P(Z \leq (50 - 48.50) / 1.96) = P(Z \leq 0.76) \approx$ **77.7 %**. The expected duration is longer than the CPM duration (47 days) because the pessimistic estimates are more spread out than the optimistic ones: development tasks tend to overrun. This finding justified holding a schedule reserve before the submission deadline.

6.6 Crashing and time-cost trade-off

Crashing shortens critical activities by adding resources at extra cost. Three activities can realistically be crashed: F, H and M. The notional crash cost assumes that the saved days are worked as paid overtime by two members, plus a helper, at the assumed effort rate of ₹250 per person-hour (Table 6.3).

Table 6.3: Crashing data (notional effort cost, ASSUMPTION)

Activity	Normal days	Crash days	Normal cost	Crash cost	Cost slope / day
F	8	6	₹16,000	₹28,000	₹6,000
H	7	5	₹14,000	₹25,000	₹5,500
M	6	5	₹12,000	₹20,000	₹8,000

Table 6.4: Time-cost trade-off: cheapest crash plan for each duration (exhaustive search)

Duration	Crash plan (days)	Extra direct cost	Indirect cost (₹3,000/day)	Total
47	none	₹0	₹1,41,000	₹1,41,000
46	H −1	₹5,500	₹1,38,000	₹1,43,500
45	H −2	₹11,000	₹1,35,000	₹1,46,000
44	F −1, H −2	₹17,000	₹1,32,000	₹1,49,000
43	F −2, H −2	₹23,000	₹1,29,000	₹1,52,000
42	F −2, H −2, M −1	₹31,000	₹1,26,000	₹1,57,000

The cheapest activity to crash is H (slope ₹5,500/day), then F, then M. Because every crash slope is higher than the assumed indirect cost of ₹3,000 per day, total cost rises steadily as the project is shortened (Table 6.4). The normal 47-day schedule is therefore the least-cost plan, and crashing is justified only if an external deadline moves earlier. For example, meeting a 45-day deadline would mean crashing H by two days for an extra ₹11,000. After H and F are crashed together, activity K becomes nearly critical, so further crashing gives diminishing returns.

CHAPTER 7

Gantt Chart and Project Timeline

Figure 7.1 presents the schedule as a Gantt chart drawn from the same CPM data. Each bar starts at the activity's earliest start; red bars are critical, and hatched extensions show total float. Milestones M1–M5 are marked as diamonds. Actual calendar dates were not fixed when the plan was made, so the chart is a planning schedule in relative weeks: week 1 begins after the project charter is approved, and the project ends in week 10 (day 47).

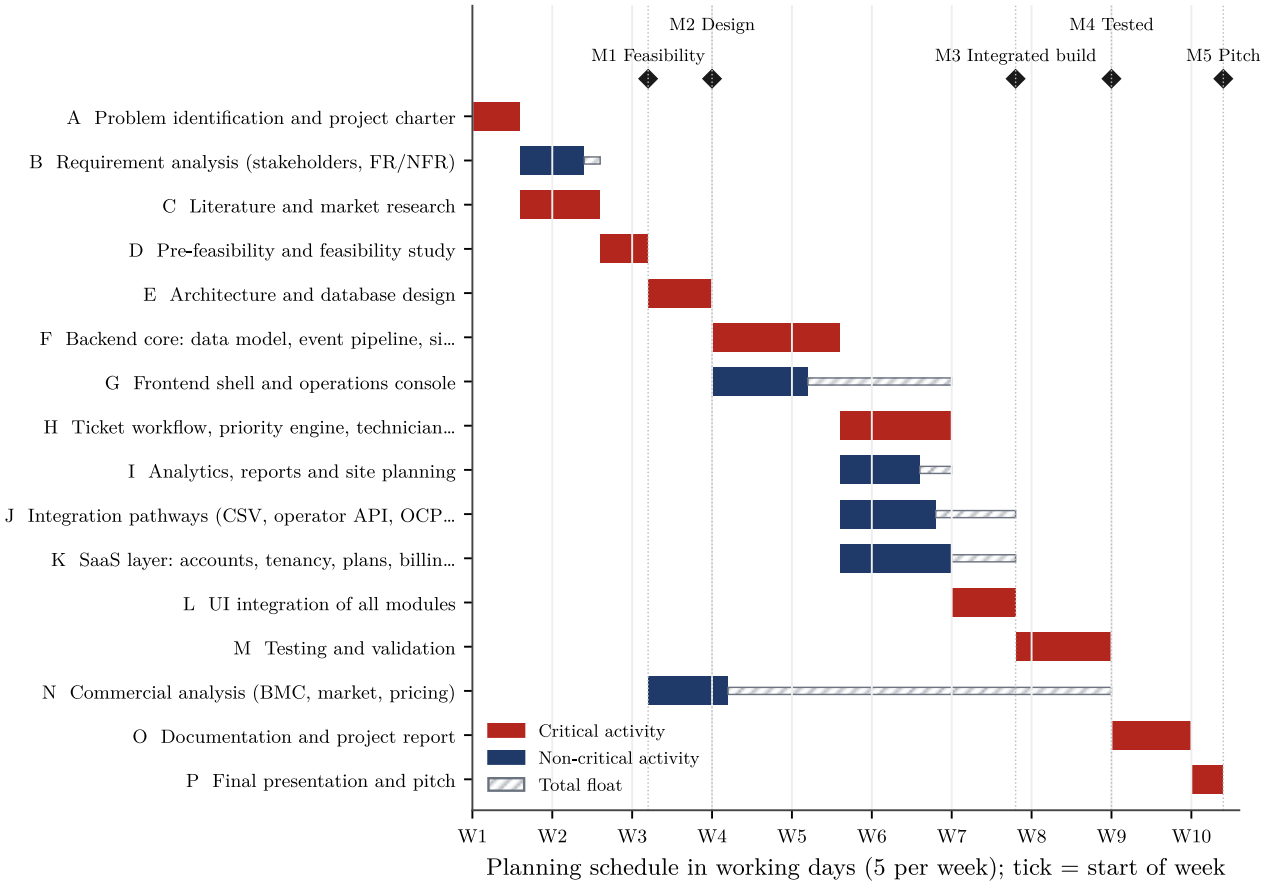


Figure 7.1: Gantt chart (planning schedule, 47 working days, 5 days per week)

Table 7.1: Milestones

Milestone	Description	End of day	Week	Completing activity
M1	Feasibility approved	11	W3	D
M2	Architecture and data model baselined	15	W3	E
M3	Integrated build	34	W7	L
M4	Tested release (all tests pass)	40	W8	M
M5	Report submitted and pitch delivered	47	W10	P

Testing (M) is scheduled as a dedicated six-day activity after integration, but unit tests were written alongside each module, so the testing activity concentrates on integration, security, tenant-isolation and browser tests. Documentation (O) depends on both testing and commercial analysis, so the report describes the tested system and the validated plan.

CHAPTER 8

Resource Requirements

8.1 Human resources and allocation

Table 8.1 allocates activities to roles; the figures are lead effort, at about 4 hours per working day. Every member also supports reviews, testing and documentation, so the plan budgets the whole team's time at 4 members × 47 days × 4 hours = 752 person-hours.

Table 8.1: Resource-allocation table (lead effort by role)

Role	Lead activities	Lead days	Lead hours
Project manager	A, D, N, O, P	18	72
Backend lead	E, F, H, J, K	32	128
Frontend and UX lead	G, I, L	15	60
Business analyst and QA lead	B, C, M	15	60

8.2 Tools, computing, data and hosting

Table 8.2: Existing resources and additional resources needed for production

Resource	Existing (prototype, no cash cost)	Additional for production
Development tools	Python 3, FastAPI, SQLAlchemy, VS Code, Git, Playwright (open source)	Code review and CI service (free tier sufficient initially)
Computing	Team members' laptops	One production and one staging cloud server
Database and storage	SQLite file with verified backups	PostgreSQL with automated backups and point-in-time recovery
Hosting and deployment	Local run (`python run.py`)	Domain, TLS certificate, reverse proxy, uptime monitoring
External data	Open Charge Map snapshot (free key), optional Google Maps key	Operator API agreements; Google Maps within free caps
Integration	Local OCPP 1.6J simulator	Real chargers or an operator sandbox for pilot testing
Testing	386 automated tests including headless-browser tests	Load testing, independent security review
Documentation	README, architecture, ticketing and SaaS documents, this report	User guide, service terms, privacy and data-processing agreement

CHAPTER 9

Detailed Cost Estimation, Budget and Capital Budgeting

9.1 Costing approach

Costs are estimated in Indian rupees and split into four groups: (a) **existing resources**, which incurred no cash cost; (b) **development effort**, valued notionally, since the team is not paid; (c) **one-time production costs**; and (d) **recurring operating costs**. Provider prices quoted in US dollars are converted at an assumed ₹88 per US$. The DigitalOcean prices were taken from its public price list [10], and the Google Maps free monthly caps from [11]. Every other figure is labelled ASSUMPTION. No paid service is in use today.

9.2 Development effort (notional)

Development effort is 4 members × 47 days × 4 h = 752 person-hours, valued at an assumed ₹250 per hour (student or intern level) = **₹1,88,000**. This is an opportunity cost used for appraisal and crashing analysis, not an expense.

9.3 One-time and recurring costs

Table 9.1: One-time production costs

Item	Amount	Basis
Domain name (1 year, .in)	₹1,000	ASSUMPTION: typical registrar price
Independent security review before pilot	₹40,000	ASSUMPTION: small-scope external review
Terms, privacy and data-processing agreement drafting	₹15,000	ASSUMPTION: legal consultation
Pilot onboarding (travel, workshops with one operator)	₹10,000	ASSUMPTION
PostgreSQL migration and deployment hardening (40 h)	₹10,000	Notional effort at the assumed rate
Total one-time	**₹76,000**	

Table 9.2: Recurring operating costs

Item	Per month	Per year	Basis
Production server: Basic Droplet 2 GiB / 1 vCPU (US$12)	₹1,056	₹12,672	DigitalOcean price list [10]
Weekly backups (20 % of Droplet price)	₹211	₹2,532	DigitalOcean price list [10]
Staging server: Basic Droplet 1 GiB (US$6)	₹528	₹6,336	DigitalOcean price list [10]
Maps: Google Maps within free monthly caps	₹0	₹0	Free usage per SKU [11]; OpenStreetMap fallback
Open Charge Map public listings	₹0	₹0	Free API key (registered)
Transactional e-mail (password reset, invitations)	₹500	₹6,000	ASSUMPTION: entry tier
Domain amortised (1,000 / 12)	₹83	₹996	From one-time cost

Item	Per month	Per year	Basis
Maintenance and support effort (20 h)	₹5,000	₹60,000	Notional effort at the assumed rate
Total recurring	**₹7,378**	**₹88,536**	**of which cash: ₹2,378/month**

Table 9.3: First-year production budget

Component	Amount
One-time costs	₹76,000
Recurring costs (12 months)	₹88,536
Contingency (10 %)	₹16,454
Total first-year budget	**₹1,80,990**
Memo: notional development effort already invested (not cash)	₹1,88,000

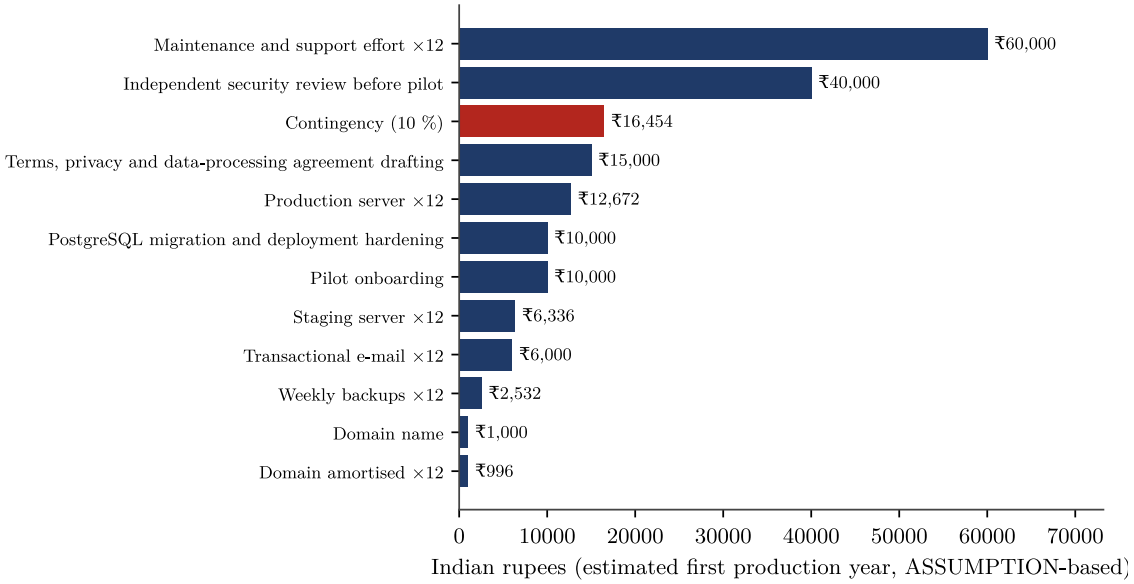


Figure 9.1: Cost breakdown for the first production year (estimated, ASSUMPTION-based)

Figure 9.1 shows that people (security review, maintenance effort) dominate the budget, not infrastructure. The cash cost of hosting is only ₹2,378 per month, because the system runs comfortably on a small server during a pilot.

9.4 Project cost-reduction methods

Following the Unit 3 cost-reduction methods, the project uses: (1) open-source software throughout, with no licence fees; (2) free tiers and free data (Open Charge Map, Google Maps free caps, with an OpenStreetMap fallback); (3) value engineering, choosing SQLite and a single deployable application for the pilot instead of a microservice architecture; (4) a simulation-first MVP, avoiding hardware purchase; (5) automated testing, which reduces rework cost; and (6) scheduling non-critical work in float periods instead of crashing.

9.5 Capital budgeting (hypothetical)

The capital-budgeting process was applied to the decision to take ChargeSight from prototype to a paid service. The cash flows in Table 9.4 are **hypothetical**. They assume that paying customers grow from 0 to 10, 25 and 45 at the end of years 1–3 (averages of 5, 17.5 and 35), that the average net revenue per

customer is ₹1,224 per month (Section 15.3), and that operating costs rise by 10 % a year.

Table 9.4: Hypothetical three-year cash flows (₹; discount rate 12 %)

Year	Avg. customers	Inflow	Outflow	Net cash flow	Cumulative
0	—	—	—	−₹76,000	−₹76,000
1	5	₹73,441	₹88,536	−₹15,095	−₹91,095
2	17.5	₹2,57,044	₹97,390	₹1,59,654	₹68,559
3	35	₹5,14,088	₹1,06,243	₹4,07,845	₹4,76,404

With these assumptions, the net present value at a 12 % discount rate is **₹3,28,094**, and the payback period is about **1.6 years**. The NPV is positive only if customer acquisition reaches the assumed levels; with half the customers in year 1, the payback period moves into year 2. These figures are an appraisal method, not a forecast, and must be re-estimated after the pilot.

9.6 Social cost-benefit analysis

Social cost-benefit analysis (SCBA) asks whether the project is worth undertaking for society, not only for its owners. **Social benefits** (qualitative; not yet measured): higher availability of public chargers means fewer failed charging visits and less range anxiety for drivers; it improves the return on public subsidies spent on charging infrastructure [5]–[7]; it supports EV adoption and lower urban emissions; and it creates skilled technician and software jobs. **Social costs and obligations**: operator and technician data must be protected (encryption, access control, tenant isolation); the electricity behind charging still carries a grid-emission footprint; and dependence on a software platform creates a continuity obligation to customers. On balance, the project shows a favourable qualitative SCBA, but measuring its benefits requires before-and-after availability data from a pilot.

CHAPTER 10

Project Monitoring and Control

10.1 Role of the project manager

The project manager owns the baseline plan and is accountable for scope, schedule, cost, quality, risk and communication. In ChargeSight the project manager chairs a weekly review, maintains the risk register and the change log, protects the critical path (F, H, L, M), approves scope changes, consolidates the MIS report, and leads the project audit at closure. The technical leads report progress on their work packages; the QA lead reports test and defect status.

10.2 Monitoring indicators and MIS

A management information system (MIS) in project monitoring collects data on progress, cost and quality, and presents it to decision-makers in time for corrective action [1], [3]. ChargeSight's project MIS is deliberately lightweight. It consists of the Git history and issue list, the automated test report (pass, fail and skip counts), a shared schedule sheet derived from Table 6.1, and a one-page weekly status report. Table 10.1 defines the indicators.

Table 10.1: Project-monitoring table

Indicator	Target / threshold	Measurement	Frequency	Corrective action
Critical-path progress	No critical activity late by > 1 day	Planned vs actual finish of A, C, D, E, F, H, L, M, O, P	Weekly	Re-assign float resources from G, J, K, N; crash H first
Schedule performance	SPI ≥ 0.95	Earned value: completed work packages ÷ planned	Weekly	Re-baseline non-critical work; reduce scope of proposed items
Effort / cost	Within ±10 % of budgeted hours	Logged hours vs plan per work package	Weekly	Review estimates; use contingency only with PM approval
Test health	0 failures, 0 skipped	Automated test report	Every change	Stop feature work until fixed (fix-first rule)
Defects	No open critical defects at a milestone	Issue list by severity	Weekly	Defect triage; add a regression test for each fix
Security and isolation	Every route guarded; isolation tests pass	Guard-coverage test, tenancy tests	Every change	Block release; review the access rule
Data honesty	No simulated data labelled LIVE	Browser and API tests on labels	Every release	Correct the label path; add a test
Scope change	All changes logged and approved	Change log	As raised	Impact analysis on schedule and cost before approval

10.3 Quality assurance, change control and progress reporting

Quality assurance combines unit tests for the pure core models, API tests for workflows, security and tenant isolation, and real-browser tests for the user interface. A change is accepted only if the whole suite passes. **Change control**: requests (for example, the later addition of a test payment step, or the separation of the Presentation Demo from customer workspaces) are logged, assessed for schedule and risk impact, approved by the project manager and recorded in the documentation together with the database migration and backup procedure. **Progress reports** summarise completed work packages, the critical-path status, test results, risks and decisions needed.

10.4 Project audit, acceptance and closure

A **project audit** is an independent review of the project's progress, performance and processes against the plan [3]. The closure audit for ChargeSight checks: (1) every deliverable against the acceptance criteria in Section 2.4; (2) the actual against the planned schedule and effort, with explained variances; (3) test evidence and open defects; (4) whether claims in the report match the repository (no unverified features, customers or results); and (5) lessons learned. The project is closed when the faculty accepts the report and presentation, the repository and documentation have been handed over, and the lessons learned have been recorded.

10.5 Issues and problems in project management

The syllabus lists common issues in project management. Those that arose or were anticipated in ChargeSight include: **scope creep** (many attractive features, such as live maps, payments and an AI prediction module), controlled by the in/out-of-scope table and the change log; **unrealistic estimates** for integration work, mitigated through PERT and a schedule reserve; **dependence on external par-**

ties (operator API access not yet granted), handled by the simulation-first design; **resource conflicts with coursework**, handled by using float; and **quality versus deadline pressure**, handled by the fix-first test rule.

CHAPTER 11

Risk Identification and Mitigation

Risks were identified through a structured review of each WBS phase and of the entrepreneurship risk-opportunity perspective in Unit 4. Likelihood (L) and impact (I) are scored from 1 (low) to 5 (high); the risk score is L × I. Scores of 15 or more are high, 8–14 medium, and below 8 low.

Table 11.1: Risk register

ID	Category	Risk	L	I	Score	Mitigation	Contingency	Owner
R1	Technical	Simulator or data model does not reflect real charger behaviour	3	4	**12**	Use OCPP 1.6 message semantics; replay imported operator data when available	Calibrate with pilot data; adjust fault model	Backend lead
R2	Integration	Operator API access or OCPP chargers unavailable for testing	4	4	**16**	Simulation-first design; mocked adapter tests; local OCPP simulator	Pilot with CSV import of operator exports	Project manager
R3	Security	Unauthorised access or session hijacking	2	5	**10**	Argon2id, CSRF tokens, deny-by-default guard, rate limits, security tests	Incident response, session revocation, external review	Backend lead
R4	Privacy	Cross-tenant data leakage	2	5	**10**	ORM-level organisation filter; isolation tests for every resource	Disable affected routes; audit and notify	Backend lead
R5	Operational	Wrong repair order harms service (poor weights)	3	3	**9**	Explainable factors; configurable weights; starvation cap	Policy lab to compare FIFO vs priority on real traces	BA / QA lead
R6	Schedule	Critical activities F or H overrun	3	4	**12**	PERT reserve; strongest developer on F/H; weekly critical-path review	Crash H then F; defer proposed scope	Project manager
R7	Financial	Paying customers fewer than assumed	4	4	**16**	Pilot before launch; low fixed costs; free Demo plan for evaluation	Pivot to service bundles with maintenance providers	Project manager
R8	Market adoption	Operators keep using vendor CPMS portals	3	4	**12**	Integrate with existing CPMS data instead of replacing it	Focus on O&M add-on for small operators	BA / QA lead
R9	Usability	Technicians find the workflow heavy in the field	3	3	**9**	Simple lifecycle actions; responsive UI	Mobile-first technician view in the next phase	Frontend lead
R10	Maintenance	Small team cannot sustain support after the course	3	3	**9**	Documentation, tests, simple architecture	Partner or hand over to an incubator team	Project manager

ID	Category	Risk	L	I	Score	Mitigation	Contingency	Owner
R11	Data quality	Imported or live data incomplete or delayed	3	3	**9**	Validation, provenance labels, stale-data states	Show data-quality warnings; never substitute simulated data	Backend lead
R12	Legal / payments	Test checkout mistaken for real billing	2	4	**8**	TEST labels everywhere; no real gateway; no card data stored	Integrate a certified gateway only after legal review	Project manager

Table 11.1 is the full risk register, and Figure 11.1 plots each risk on a likelihood-impact heat map. The highest exposures are integration access (R2), customer acquisition (R7), market adoption (R8) and schedule (R6). These are business and dependency risks rather than code risks, which matches the prototype stage of the project.

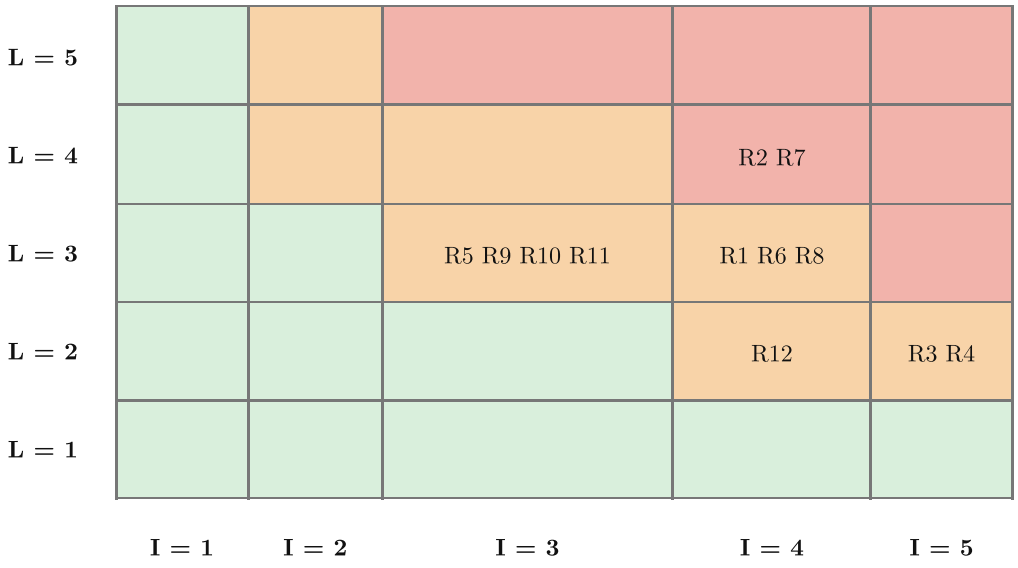


Figure 11.1: Risk heat map (likelihood × impact; red ≥ 15, orange 8–14, green < 8)

11.1 Risk-opportunity perspective

Unit 4 frames entrepreneurial risk together with opportunity. The dependency on operator data (R2) is also the main opportunity: an operator that shares data in a pilot gains a free operations-and-maintenance improvement, and ChargeSight gains validation. Similarly, the subsidy-driven expansion of public charging [6], [7] raises the stakes of downtime and so strengthens the case for maintenance tooling. Each high risk is therefore paired with a deliberate learning step in the pilot plan (Chapter 13).

CHAPTER 12

Business Model Canvas

The Business Model Canvas (Figure 12.1) summarises how ChargeSight would create, deliver and capture value. Every element is a hypothesis to be tested in customer discovery; nothing in it implies existing customers, partners or revenue.

8. Key partnerships
- Charger manufacturers and CPMS vendors (data access, OCPP)
- Maintenance and field-service contractors
- Site hosts (malls, offices, fuel outlets)
- Cloud and mapping providers
- College incubation centre, EV industry bodies

7. Key activities
- Software development and testing
- Integrations with operator data and OCPP
- Customer onboarding and support
- Security, privacy and compliance

6. Key resources
- Priority, assignment and SLA logic
- Multi-tenant SaaS platform and tests
- Domain knowledge of charger faults
- Development team

2. Value propositions
- One view of charger health across data sources
- Every fault becomes one accountable ticket
- Explainable repair priority (factor by factor)
- Right technician, within SLA, with an audit trail
- Uptime, MTTA, MTTR and site-planning evidence
- Affordable for small operators; free Demo plan

4. Customer relationships
- Self-service onboarding with guided setup
- Pilot partnership with close support
- Dedicated account management for Enterprise

3. Channels
- Direct outreach to operators and fleet depots
- Public website with free Presentation Demo
- Partner referrals (CPMS vendors, installers)
- EV events and incubator networks

1. Customer segments
- Small and mid-sized charge-point operators
- EV fleet operators with depot charging
- Charging-infrastructure maintenance providers
- Site owners running their own chargers
- (All subject to validation)

9. Cost structure
- Hosting and backups (≈ ₹2,378/month cash, Chapter 9)
- Development, maintenance and support effort
- Security review, legal documents
- Customer acquisition and pilot onboarding

5. Revenue streams
- Monthly or annual subscriptions: Starter ₹499, Growth ₹1,499, Professional ₹4,999 per month (proposed)
- Commitment discounts of 5 % (6 months) and 10 % (12 months)
- Enterprise: negotiated contracts
- Optional: onboarding and integration services

Figure 12.1: Business Model Canvas for ChargeSight (hypotheses to be validated)

12.1 Delivering measurable value without unverified claims

ChargeSight does not claim a percentage saving or uptime gain, because no field data exists yet. Instead, it makes value measurable: the pilot will record baseline availability, MTTA, MTTR and overdue-ticket counts before ChargeSight is adopted, and then compare them with the same metrics afterwards. The value proposition is thus expressed as indicators the customer can verify in their own data.

CHAPTER 13

Market, Industry and Customer Analysis

13.1 Market segment and target customers

Market segmentation (Unit 5) divides the market by operator size and charger ownership. The primary target segment is small and mid-sized charge-point operators that run tens to a few hundred public chargers and lack a dedicated network-operations team. Secondary segments are fleet operators with depot charging, and maintenance contractors serving several operators. Large national networks are a later Enterprise segment.

Table 13.1: Buyer and user personas (illustrative, not interview results)

Persona	Role	Goals	Pain points
Operations head of a small CPO (buyer)	Owner / operations manager	High availability, predictable maintenance cost, reports for site hosts	Scattered status data, repair order based on complaints, no SLA evidence
Field technician (user)	Technician	Clear jobs, fewer wasted trips, safe procedures	Unclear fault details, jobs outside their skill or equipment
Fleet depot manager (buyer)	Fleet operator	Vehicles charged on time every night	A failed depot charger blocks vehicles the next morning

13.2 Industry and competitor analysis

Market structure. Charging software is a competitive market with global CPMS platforms, hardware companies that bundle software, and in-house tools at large operators. **Market size and growth potential.** The installed base of more than 29,000 public stations [5] and the target of about 72,000 additional subsidised stations under PM E-DRIVE [6] indicate a growing pool of chargers that need maintenance. The team has not estimated a rupee market size, because no reliable figure for operator software spending was available; the illustrative calculation below is explicitly hypothetical. **Demand-supply analysis.** Demand for maintenance tooling grows with every charger installed and with the subsidy-linked expectation of availability. On the supply side, comprehensive platforms exist but are positioned for large operators, which leaves room for an affordable O&M-focused tool.

Table 13.2: Existing alternatives and positioning (based on public product information)

Alternative	What it offers (public sources)	Gap ChargeSight targets
AMPECO [13]	CPMS with remote maintenance, real-time network monitoring, asset configuration, operational and financial reporting; OCPP support	Explainable, factor-level repair prioritisation and technician eligibility rules are not described on the O&M page
Driivz (Vontier) [14]	Cloud platform for charging operations, energy management, billing and driver self-service; customers in 30+ countries	Positioned for large operators; ChargeSight targets small operators at low price points
Kazam [15]	Indian maker of light-EV AC/DC chargers with software to manage chargers and hubs	ChargeSight is hardware-agnostic and focused on maintenance workflow
Spreadsheets and messaging groups	Free, flexible	No audit trail, prioritisation, SLA or analytics
Generic field-service tools	Work orders and scheduling	No charger status, OCPP events or fault semantics

13.3 Factors determining competitive advantage

ChargeSight's intended competitive advantage rests on four factors: (1) **explainability** — every rank comes with factor contributions, which builds trust with site hosts and managers; (2) **maintenance depth** — technician eligibility, safety inspection before return to service, SLA escalation and recurrence reopening; (3) **honest data handling** — strict provenance labels and tenant isolation; and (4) **low cost of adoption** — a free Demo plan, a ₹499 entry plan and CSV import without integration work. Factor (1) can be imitated, so a lasting advantage will depend on execution, integrations and customer relationships.

13.4 Customer discovery, adoption barriers, pilot and market entry

Customer discovery plan. The team will interview at least 10 operators, fleet managers and technicians in Bengaluru, using the personas above as hypotheses to test. Each interview will check the six problem hypotheses in Section 1.2, current tools and willingness to pay. The validation module in the prototype will record this evidence. **No interviews have been conducted yet**, and an operator API request to one Indian company is still pending. **Adoption barriers**: data-sharing permission, integration effort, trust in a student-built product, and switching costs from existing portals. **Pilot strategy**: one operator, 8–12 weeks, read-only data through a CSV export or API, with baseline and after metrics (availability, MTTA, MTTR, overdue tickets) and a written success criterion agreed in advance. **Market-entry approach**: start with Bengaluru operators, enter through a free pilot and the Starter/Growth plans, and grow through referrals from installers and maintenance contractors.

> **Illustrative only.** If 1 % of the 29,000+ public stations [5] belonged to operators that adopted a Growth-level plan, and each operator ran about 15 stations, the result would be roughly 19 customers. This shows the order of magnitude of a first local target. It is not a market estimate.

CHAPTER 14

Prototype (MVP) and Validation

14.1 Minimum viable product

The MVP is the smallest product that lets an operator run the core maintenance loop end to end: see the fleet → receive a ticket → understand its priority → assign a qualified technician → track it to closure within SLA → review the metrics (Figure 14.1). Everything in this loop is implemented and demonstrable in the Presentation Demo workspace: a separate, per-user workspace with 6 demonstration sites, 40 SIMULATED chargers, 4 fictional technicians, 6 hours of history and 8 scripted tickets covering every lifecycle state.

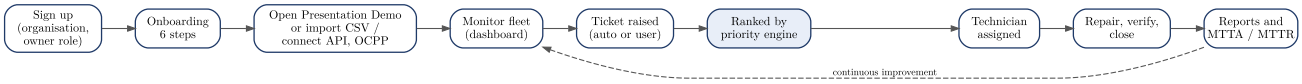


Figure 14.1: MVP user journey

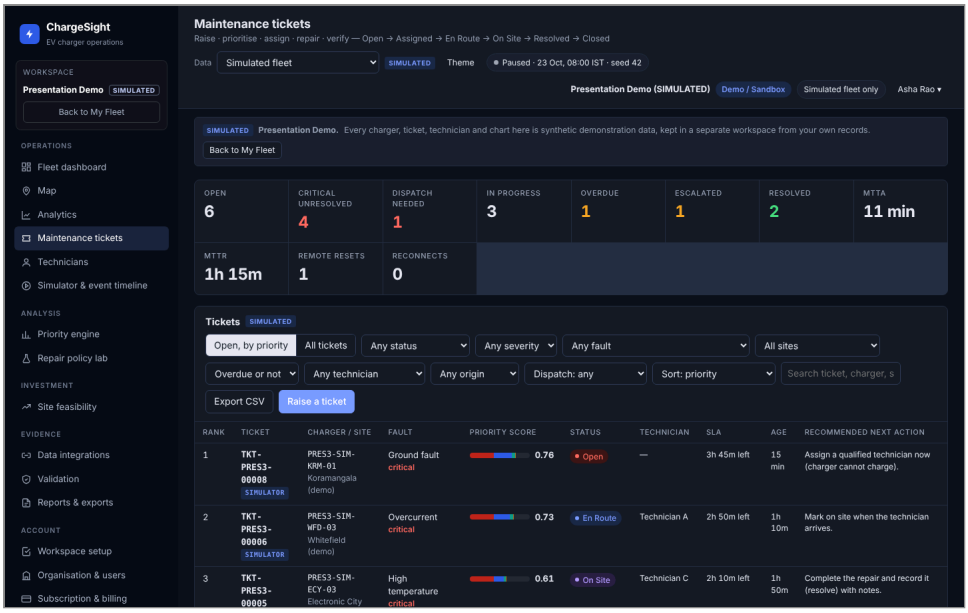


Figure 14.2: Ranked maintenance tickets with priority scores, technician, SLA and next action (real screenshot; SIMULATED data)

14.2 Verified functionality versus planned functionality

Table 14.1: MVP feature status

Capability	Status	Evidence
Core dashboard, charger status and history charts	Implemented	Presentation Demo; browser tests
Maintenance tickets, lifecycle and audit history	Implemented	Ticketing test suite
Explainable priority and starvation cap	Implemented	Priority unit tests; per-ticket factors
Technician eligibility and assignment	Implemented	Assignment tests (skills, AC/DC, safety, capacity)
SLA deadlines and escalation	Implemented	SLA tests (single escalation)
Reports and CSV exports	Implemented	Export tests with provenance metadata
Operator API / OCPP integration	Implemented, not live	Mocked API tests; local OCPP WebSocket handshake
Accounts, roles, tenant isolation	Implemented	Authentication and tenancy test suites
Plans, entitlements, TEST checkout	Implemented (no real payments)	Billing and checkout tests
Mobile technician app, notifications by SMS / e-mail	Planned	—
Failure prediction (machine learning)	Planned (future scope)	—

14.3 Functional, security and tenant-isolation testing

The latest full run of the automated suite (`python -m pytest`) passed 385 tests, with 0 failures and 0 skipped, in about 15 minutes. One further test, added during that run, passed when run on its own, bringing the total to 386. The suite includes 20 tests that drive a real headless browser. The tests cover the priority formula, simulation determinism, the ticket workflow and SLA, technician constraints, policy

simulation, the finance model, integrations (including a real local OCPP 1.6J handshake), authentication (password hashing, CSRF, rate limits, session expiry), tenant isolation (cross-organisation identifiers in URLs and bodies return 404), plan limits, and the TEST checkout. The checkout tests confirm that no card number, CVV, OTP or PIN is ever stored, and that no paid plan can be activated without the payment step. Appendix B lists the verified facts. The tests use mocks for external services; they are not evidence of a live operator deployment.

14.4 Remaining gaps before commercial deployment

- Validation with real operator data and a pilot (no live operator connected yet).
- PostgreSQL migration, load testing and production monitoring.
- An independent security review; privacy policy and data-processing agreement.
- A certified payment gateway with GST invoicing (the checkout is TEST-only today).
- Notifications (e-mail/SMS) and a mobile-optimised technician view.
- Calibration of the priority weights and SLA hours with operators.

CHAPTER 15

Entrepreneurship, Commercial Viability and Pitch

15.1 Entrepreneurship and innovation in ChargeSight

Entrepreneurship is the process of identifying an opportunity and organising resources to create value from it while bearing risk; innovation is the creation and adoption of something new that adds value [1], [2]. ChargeSight is a **process innovation** for charging operators: it does not invent a new charger, but changes how maintenance decisions are made and evidenced. Its potential contribution to society lies in more reliable public charging, better use of public subsidies, and skilled employment (Section 9.6). The **qualities of a prospective entrepreneur** listed in Unit 4 — initiative, risk-taking within limits, persistence, learning orientation and integrity — are reflected in the team's choices to build a working prototype before asking for data, and to label every assumption.

15.2 Innovation management, idea management and thinking styles

Table 15.1: Steps of innovation management applied to ChargeSight

Step	ChargeSight
Idea generation (divergent thinking)	Candidate ideas included charger uptime monitoring, a charger-finder app, battery-swap scheduling, fleet charging planning and predictive maintenance
Idea screening (convergent thinking)	Ideas were scored on problem evidence, MVP feasibility without hardware, and team skills; uptime monitoring with repair prioritisation was selected
Concept development	Problem statement, personas, the priority formula and the workflow (Chapters 1–3)
Prototyping	Simulation-first MVP with tests (Chapter 14)
Validation	Customer discovery and pilot plan (Chapter 13)
Commercialisation	Business model, pricing hypotheses, phased launch (this chapter)

An **idea management system** captures, evaluates and tracks ideas. In this project, the issue list and the change log served that purpose: every proposed feature was recorded with its rationale, assessed against scope, and either scheduled or explicitly deferred. **Challenges of innovation** faced here include access to real data, user trust and the risk of building features nobody needs; the validation-first plan addresses all three.

15.3 Revenue model, pricing hypotheses and plan differentiation

Table 15.2: Proposed subscription catalogue as implemented in the prototype (proposed prices, not validated)

Plan	Price / month	Managed chargers	Users	Sites	Integrations	Differentiating features
Demo / Sandbox	Free	Simulated only	2	—	—	Explore with SIMULATED data
Starter (recommended entry)	₹499	5	2	1	—	Dashboard, tickets, priority, manual assignment, basic SLA and analytics, CSV import
Growth	₹1,499	25	5	3	3	Auto-assignment, SLA escalation, advanced analytics, site planning, policy lab
Professional	₹4,999	100	15	10	10	Expanded reports, enhanced administration and audit
Enterprise	Negotiated	Negotiated	Negotiated	Negotiated	Negotiated	Contract terms and support

Six-month and twelve-month commitments receive 5 % and 10 % discounts, calculated on the server (for example, Starter costs ₹2,844.30 for six months and ₹5,389.20 for twelve months). The prototype's checkout is a clearly labelled TEST flow: it accepts only published test card numbers, `@test` UPI IDs and fictional test banks, never asks for a CVV, OTP or PIN, and issues receipts marked "no payment was taken, not a tax invoice". **A simulated checkout is not evidence of revenue.**

Unit economics (hypothetical). With an assumed customer mix of 60 % Starter, 30 % Growth and 10 % Professional, the average revenue per customer is ₹1,249 per month, or ₹1,224 after an assumed 2 % payment-gateway fee. Against monthly costs of ₹7,378 (including notional support effort), break-even requires **7 paying customers** (Figure 15.1). Covering the cash costs alone would need 2.

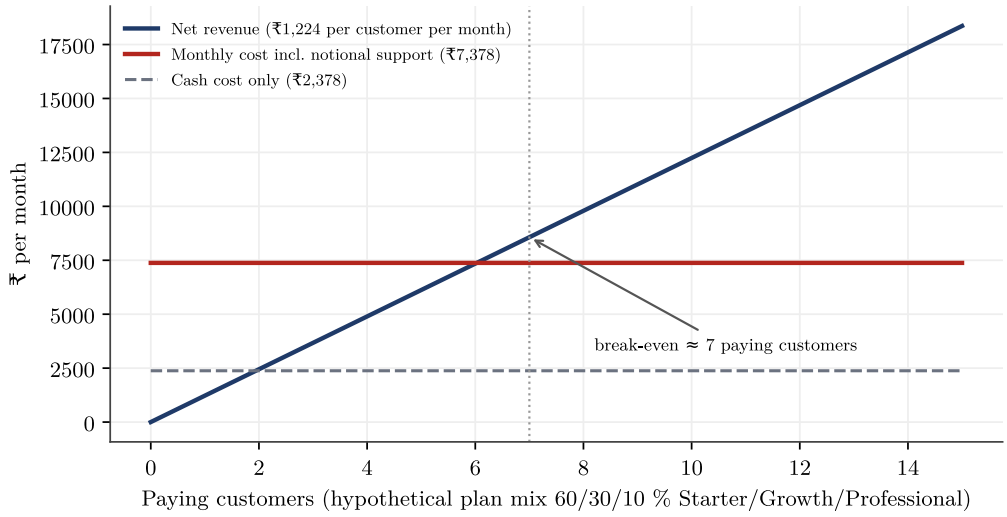


Figure 15.1: Break-even chart (hypothetical; based on proposed prices and an assumed plan mix)

15.4 Customer acquisition, partnerships, scaling and competitive risks

Acquisition: founder-led outreach to Bengaluru operators, free Presentation Demo, pilot-to-paid conversion, and referrals from installers and maintenance contractors. **Partnerships and support**: data access agreements with charger or CPMS vendors, a maintenance-contractor partner for field services, and an incubator for mentoring and legal support. **Scaling**: PostgreSQL and horizontal scaling of the stateless API, more integrations, a mobile technician view, then regional expansion. **Competitive risks**: incumbents could add explainable prioritisation; operators may prefer bundled vendor software; pricing may need to change after validation. **Commercial validation** milestones: (1) pilot agreement signed; (2) baseline metrics captured; (3) improvement measured; (4) first paid conversion; (5) three referenceable customers.

15.5 Idea incubation, design thinking and TRIZ

Idea incubation in Unit 5 covers nurturing an idea into a viable venture. ChargeSight's next step is a college incubation or pre-incubation programme, where mentoring, a pilot introduction and legal support would address risks R2, R7 and R10. **Design thinking** guided the product: *empathise* (operator, technician and driver needs, Section 1.4), *define* (the problem statement), *ideate* (ideas screened in Table 15.1), *prototype* (the simulation-first MVP) and *test* (automated tests now; user testing in the pilot). **TRIZ** resolves technical contradictions with inventive principles. Two contradictions were addressed: (1) *respond quickly to every fault* versus *limited technicians*, resolved through *prior action* (an automatic remote reset before dispatch) and *segmentation* (assigning only eligible technicians by skill and equipment); and (2) *realistic demonstration* versus *no access to real chargers*, resolved through *copying* (a deterministic simulator that emits OCPP-style events through the same pipeline as live data).

15.6 Entrepreneurial pitch

The problem. India already has more than 29,000 public charging stations, and public money is funding tens of thousands more. A charger that does not work fails the driver, the site host and the investment behind it, and small operators still manage repairs through calls and spreadsheets.

Our solution. ChargeSight turns every charger fault into one accountable ticket, ranks it with a priority score anyone can read, sends a technician who is qualified and available, and tracks the repair against an SLA, with the evidence saved for every decision.

Why us, why now. A working, tested multi-tenant prototype exists today: 176 secured API routes, 386 passing tests, OCPP 1.6J and operator API pathways, and a free demo anyone can open. Pricing starts at ₹499 a month, within reach of small operators.

What we ask. One operator willing to run an 8–12-week pilot with read-only data, so that we can measure availability, MTTA and MTTR before and after adoption, and support from an incubator to turn the pilot into a product.

CHAPTER 16

Expected Outcomes, Conclusion and Future Scope

16.1 Expected outcomes

Engineering contribution. A tested, multi-organisation SaaS prototype that implements the full maintenance loop for EV chargers, with explainable prioritisation, rule-based assignment, SLA control, analytics and honest provenance labelling. **Project-management learning.** The team applied the life cycle, feasibility appraisal, WBS/OBS, CPM and PERT (a 47-day critical path, with a 77.7 % probability of finishing within 50 days), crashing and time-cost trade-off, cost estimation, capital budgeting, monitoring with an MIS, project audit and risk management to a real software project. **Operational benefits (expected, to be measured).** Faster detection, a defensible repair order, fewer misassigned visits and measurable SLA performance. **Technical feasibility** has been demonstrated with the prototype. **Commercial potential** is plausible but unvalidated: break-even is hypothetical, at about 7 paying customers.

16.2 Conclusion

ChargeSight shows that a small team can build a credible operations and maintenance platform for EV charging when the work is scoped, scheduled and controlled with sound project-management methods. The prototype is deliberately honest about its limits: its fleet data is simulated, its integrations are tested but not live, and its checkout takes no payment. The analysis identifies the next steps that matter most — customer discovery and a pilot with real data — and quantifies the effort and cost needed to reach them.

16.3 Current limitations and future scope

- **Limitations:** no real operator data or field validation; priority weights and SLA hours are assumptions; SQLite and a single server are suitable only for a pilot; no notifications or real payments; no formal usability study.
- **Next steps (validation and production readiness):** customer interviews; a pilot with baseline metrics; PostgreSQL migration and load testing; an independent security review; a certified payment gateway with GST invoicing; e-mail/SMS notifications; a mobile technician view.
- **Future scope:** failure prediction from pilot history (machine learning, after enough labelled data exists), spare-parts planning, route-optimised dispatch, energy-aware scheduling, and integration with additional CPMS platforms.

References

[1] S. N. Banerjee, *Entrepreneurship and Project Management.* Thakur Publications Pvt. Ltd. (Prescribed text book, 23IS7PCPME.)

[2] S. S. Khanka, *Entrepreneurship Development.* New Delhi: S. Chand & Co. (Prescribed text book, 23IS7PCPME.)

[3] C. F. Gray, E. W. Larson and G. V. Desai, *Project Management: The Managerial Process.* McGraw Hill Education. (Prescribed text book, 23IS7PCPME.)

[4] B.M.S. College of Engineering, Department of ISE, "Course syllabus: Project Management, Entrepreneurship (23IS7PCPME)," 2026–2027.

[5] News On AIR (Akashvani), "Over 29,000 public EV charging stations installed in India: MoS Bhupathiraju Srinivasa Varma," Feb. 10, 2026. [Online]. Available: https://www.newsonair.gov.in/over-29000-public-ev-charging-stations-installed-in-india-mos-bhupathiraju-srinivasa-varma

[6] Energetica India, "Government allocates INR 2000 crore for installation of 72,000 EV charging stations under PM E-DRIVE," May 23, 2025. [Online]. Available: https://www.energetica-india.net/news/government-allocates-inr-2000-crore-for-installation-of-72000-ev-charging-stations-under-pm-e-drive

[7] Business Today, "Govt unveils ₹2,000 crore plan to boost EV charging under PM E-DRIVE," Sep. 28, 2025. [Online]. Available: https://www.businesstoday.in/india/story/govt-unveils-rs2000-crore-plan-to-boost-ev-charging-under-pm-e-drive-496055-2025-09-28

[8] D. Rempel, C. Cullen, M. M. Bryan and G. V. Cezar, "Reliability of open public electric vehicle direct current fast chargers," arXiv:2203.16372, Mar. 2022.

[9] J.D. Power, "2025 U.S. Electric Vehicle Experience (EVX) Public Charging Study," press release via Automotive World, 2025. [Online]. Available: https://www.automotiveworld.com/news-releases/fewer-failed-charging-attempts-signal-progress-in-ev-infrastructure-j-d-power-finds/

[10] DigitalOcean, "Droplet pricing," accessed Oct. 25, 2026. [Online]. Available: https://www.digitalocean.com/pricing/droplets

[11] 9to5Google, "Google Maps Platform replaces $200 monthly credit with free usage limits per product," Dec. 9, 2024. [Online]. Available: https://9to5google.com/2024/12/09/google-maps-platform-usage-limits/

[12] Open Charge Alliance, "Introduction to OCPP," 2026. [Online]. Available: https://openchargealliance.org/wp-content/uploads/2026/04/Introduction-to-OCPP.pdf

[13] AMPECO, "EV charging platform: operations and maintenance," accessed Oct. 25, 2026. [Online]. Available: https://www.ampeco.com/us/ev-charging-platform

[14] Vontier / Driivz, "Vontier's Driivz releases Version 8 of smart EV charging and energy management software platform," Nov. 12, 2024. [Online]. Available: https://www.nasdaq.com/press-release/vontiers-driivz-releases-version-8-smart-ev-charging-and-energy-management-software

[15] Electronics For You, "Kazam simplifies EV charging with innovative hardware and software solutions for all," Jan. 31, 2025. [Online]. Available: https://www.electronicsforu.com/electronics-startups/kazam-simplifies-ev-charging-with-innovative-hardware-and-software-solutions-for-all

[16] EV ChargeSight project repository and documentation (README, docs/ARCHITECTURE.md, docs/TICKETING.md, docs/SAAS.md), B.M.S. College of Engineering, 2026.

Appendix A: Syllabus Coverage Map

The table maps each topic of the official 23IS7PCPME syllabus [4] to the sections of this report where it is applied to ChargeSight.

Unit	Syllabus topics	Report sections
1	Definitions of project and project management; issues and problems; project life cycle (initiation, planning, execution, closure)	2.5, 10.5
2	Pre-feasibility and feasibility studies; DPR; technical, economic, commercial and financial appraisal; capital budgeting; social cost-benefit analysis	2.6, 9.5, 9.6; this report as DPR
2	Importance and steps of project planning; project scope; WBS and OBS; phased project planning	5.1–5.4, 2.3

Unit	Syllabus topics	Report sections
3	Gantt chart; CPM and PERT; critical path and its significance; floats and slacks; crashing; time-cost trade-off	6.1–6.6, Chapter 7
3	Project cost-reduction methods	9.4
3	Monitoring and control: role of the project manager; MIS in project monitoring; project audit	10.1–10.4
4	Meaning and concept of entrepreneurship; innovation and entrepreneurship; contributions of entrepreneurs to society	15.1
4	Risk-opportunity perspective and mitigation of risks	Chapter 11, 11.1
4	Challenges of innovation; steps of innovation management; idea management system; divergent vs convergent thinking; qualities of a prospective entrepreneur	15.1, 15.2
5	Idea incubation; factors determining competitive advantage	15.5, 13.3
5	Market segment; industry and competitor analysis (market structure, size, growth potential); demand-supply analysis	13.1, 13.2
5	Entrepreneurial motivation: design-thinking-driven innovation; TRIZ	15.5

Appendix B: Verified Prototype Facts and Test Evidence

The facts below were verified in the project repository [16] when this report was prepared.

Item	Verified fact
Backend	FastAPI, Pydantic v2, SQLAlchemy 2.0, SQLite (WAL); about 16,100 lines of Python in the application
Frontend	Vanilla JavaScript ES modules (about 5,900 lines), SVG charts, no build step
API surface	176 routes, each with an explicit guard rule (session, CSRF, role, plan); deny by default
Data model	33 tables; provenance labels SIMULATED / IMPORTED / LIVE
Synthetic fleet	40 chargers at 6 Bengaluru demonstration areas; seed 42; 15-second simulation steps; 6 h history; 8 scripted tickets
Priority	$P = 0.4S + 0.3U + 0.2R + 0.1A$; 24 h age cap and 24 h starvation cap (design assumptions)
SLA and roles	Critical 4 h, major 12 h, minor 48 h (assumptions), single escalation; roles: owner, admin, operations manager, technician, viewer
Plans	Demo ₹0, Starter ₹499, Growth ₹1,499, Professional ₹4,999 per month, Enterprise negotiated (proposed prices)
Tests	386 automated tests; latest full run 385 passed, 0 failed, 0 skipped (plus 1 test added during the run, passed separately); 20 headless-browser tests
Not claimed	No live operator connection, no real customers, no revenue, no payments taken, no field validation of priority weights